

AI-Powered Health Insurance Actuarial Intelligence Platform

End-to-End Practical Implementation Guide

Team 4 · AI-AIP · June 2026

6 Modules	7 Sprint Days	10 Deliverables	5 ML Models
---------------------------------	-------------------------------------	---------------------------------------	-----------------------------------

1. Project Overview & Architecture

Problem Statement

Health insurers operate fragmented systems for underwriting, pricing, portfolio monitoring, and retention — leading to inconsistent decisions, delayed risk signals, and inefficient reporting. This platform consolidates every stage of the actuarial lifecycle into a single AI-powered decision-support system.

System Architecture — Module Flow

Data Ingestion	Feature Engineering	Risk Classification
Claim Cost Prediction	Premium Recommendation	Portfolio Monitoring
Lapse Prediction	AI Commentary	Dashboard & Reporting

Objectives

- Assess applicant risk using ML-driven underwriting models
- Predict individual claim costs with XGBoost regression
- Recommend actuarially fair premiums with configurable loading factors
- Monitor portfolio KPIs: frequency, severity, loss ratio, trend
- Predict policyholder lapse probability and flag retention actions
- Generate AI-powered executive commentary via Claude/OpenAI API
- Deliver a unified Streamlit dashboard for all stakeholders

Technology Stack

Layer	Tools
Data & processing	Python, Pandas, NumPy
ML modelling	Scikit-learn, XGBoost
Explainability	SHAP (TreeExplainer)
AI copilot	Claude API / OpenAI API
Visualisation	Plotly, Matplotlib
Dashboard	Streamlit
Version control	GitHub

2. Pre-Sprint Setup (Day 0)

Owner: Veer (Project Lead) — Complete all items below before Day 1 kick-off.

Repository Structure

Create the following folder layout in your GitHub repo:

Folder / File	Purpose
/data/raw/	Original downloaded dataset
/data/processed/	Cleaned and feature-engineered CSVs
/models/	Saved .pkl XGBoost model files
/notebooks/	Jupyter notebooks per module (EDA, modelling)
/outputs/	All CSV deliverables + executive_summary.txt
/dashboard/	Streamlit app files
run_all.py	End-to-end pipeline runner
requirements.txt	All Python dependencies
.env.example	API key template (Claude / OpenAI)
README.md	Setup & run instructions

Dataset Sourcing

Use one of the following publicly available datasets:

- **Kaggle — Medical Cost Personal Dataset:** age, sex, BMI, children, smoker, region, charges. Simple baseline.
- **Kaggle — Health Insurance Cross-Sell Prediction:** vehicle insurance upgrade data; good for lapse modelling.
- **Synthea:** Open-source synthetic patient generator. Run locally to produce realistic claims data at scale.
- **MIMIC-III (PhysioNet):** Real de-identified clinical data. Requires credentialing — use only if pre-approved.

Minimum viable schema (30+ k rows): age, gender, BMI, smoker_flag, num_dependents, region, pre_existing_conditions (list), annual_income, policy_tenure_years, previous_claims_count, previous_claims_amount, hospitalisation_flag, claim_cost (target for pricing), risk_class (target for underwriting), lapsed (target for lapse).

Shared Schema Agreement

On Day 0, Veer circulates a **feature_dict.md** file that locks in column names, data types, and allowed values. All members must use this schema for their module outputs — mismatched columns are the #1 integration failure point.

requirements.txt (starter)

```
pandas>=2.0
numpy>=1.24
scikit-learn>=1.3
xgboost>=2.0
shap>=0.44
streamlit>=1.32
plotly>=5.18
anthropologic>=0.25 # or openai>=1.0
python-dotenv>=1.0
reportlab>=4.0
jupyterlab>=4.0
```

3. 7-Day Sprint — Detailed Plan of Action

Day 1

Data Exploration & Cleaning

Veer (Lead)

- Download dataset and place in /data/raw/
- Audit all columns — note dtypes, nulls, cardinalities
- Remove exact duplicates; flag near-duplicates for review
- Document missing-value strategy: median impute for numerics, 'Unknown' for categoricals
- Produce data_quality_report.md listing all issues

Deliverable: [raw_data.csv](#)

Member 6 (Validation)

- Univariate distributions for every numeric feature (histogram + box plot)
- Bivariate: claim_cost vs smoker, BMI, age, pre-existing conditions
- Correlation heatmap — flag multicollinear pairs ($r > 0.85$)
- Save EDA notebook to /notebooks/01_eda.ipynb

Deliverable: [eda_notebook.ipynb](#)

Day 2

Feature Engineering

Veer (Lead)

- BMI bands: Underweight / Normal / Overweight / Obese
- Age groups: 18-25 / 26-35 / 36-50 / 51-65 / 65+
- Claim frequency: previous_claims_count / policy_tenure_years
- High-risk flag: smoker AND (BMI > 30 OR pre_existing > 1)
- One-hot encode: gender, region, bmi_band, age_group
- StandardScaler on: age, BMI, annual_income, claim amounts
- Train/test split: 80/20 stratified on risk_class
- Save to /data/processed/processed_data.csv

Deliverable: [processed_data.csv](#)

Member 2 (Underwriting)

- Define interaction terms: age * pre_existing_conditions
- Create condition_severity_score (weighted sum of conditions)
- Validate feature importance signal vs risk_class with quick RF

Day 3**Underwriting Risk Engine****Member 2 (Underwriting)**

- Target: risk_class (Low=0, Medium=1, High=2, VeryHigh=3)
- Model: XGBoostClassifier — n_estimators=300, max_depth=6
- Tune with 5-fold GridSearchCV on: learning_rate, subsample, colsample_bytree
- Risk score (0-100) = predict_proba weighted sum * 100
- SHAP TreeExplainer — global + per-applicant waterfall plots
- Segment output: Standard / Rated / Decline flags
- Save model to /models/underwriting_model.pkl
- Save output to /outputs/applicant_risk_scores.csv

Deliverable: [applicant_risk_scores.csv](#)

Member 6 (Validation)

- Confusion matrix + classification report (precision, recall, F1)
- AUC-ROC curve (multi-class OvR)
- Benchmark: compare XGBoost vs Logistic Regression baseline
- Bias audit: check accuracy parity across age group and gender

Day 4**Pricing, Portfolio Analytics & Lapse Models****Member 3 (Pricing)**

- Target: claim_cost (continuous — use XGBoostRegressor)
- Tune: n_estimators=400, learning_rate=0.05, max_depth=5
- Evaluate: RMSE, MAE, R-squared on test set
- Premium = predicted_claim_cost * loading_factor (default 1.25)
- Loading factors by risk band: Low=1.15, Med=1.25, High=1.45, VH=1.65
- Save to /outputs/predicted_claim_cost.csv & premium_quotes.csv

Deliverable: [premium_quotes.csv](#)

Member 4 (Portfolio)

- Claim frequency by region, age group, policy type
- Average severity by cohort — Plotly bar + box charts
- Loss ratio = total claims / total premiums — target 75-90%
- 12-month rolling trend (if date column present)
- Save summary to /outputs/portfolio_metrics.csv

Deliverable: [portfolio_metrics.csv](#)

Member 5 (Lapse)

- Target: lapsed (binary 0/1)

- Model: XGBoostClassifier with scale_pos_weight for imbalance
- Segments: At-risk (prob > 0.7), Watch (0.4-0.7), Stable (< 0.4)
- SHAP: top 5 lapse drivers per segment
- Retention flags: discount_offer / agent_call / loyalty_reward
- Save to /outputs/lapse_risk_scores.csv

Deliverable: [lapse_risk_scores.csv](#)

Day 5

Validation & Integration Testing

Member 6 (Validation)

- Run complete pipeline on holdout test set (unseen during training)
- Schema check: all CSVs match agreed column names and dtypes
- Cross-module logic: high-risk applicants should have higher premiums
- Lapse x Risk correlation: do high-lapse segments skew toward high-risk?
- Loss ratio sanity: confirm within 60-110% range
- Document all anomalies in validation_report.md

Deliverable: [validation_report.md](#)

All Members

- Each member reviews another member's module outputs
- Flag any outliers, impossible values, or schema mismatches
- Confirm model .pkl files are saved and loadable
- Merge all branches to main on GitHub

Day 6

AI Copilot & Streamlit Dashboard

Veer (Lead) — AI Copilot

- Set up .env with ANTHROPIC_API_KEY or OPENAI_API_KEY
- System prompt: 'You are an actuarial analyst. Given portfolio metrics, generate a concise executive summary.'
- Pass portfolio_metrics.csv as structured context in user message
- Parse response and write to /outputs/executive_summary.txt
- Add natural language query interface: user types question, copilot answers

Deliverable: [executive_summary.txt](#)

Veer (Lead) — Dashboard

- Page 1 — Applicant risk assessment: input form + risk score gauge
- Page 2 — Premium quote: sliders for loading factor + output table
- Page 3 — Portfolio KPIs: Plotly frequency/severity/loss-ratio charts
- Page 4 — Lapse heatmap: segment table + retention action flags

- Page 5 — AI Copilot: text input + streaming response display
- Sidebar: file upload (CSV) to run any applicant through models live

Deliverable: `app.py`

Day 7

Integration, Documentation & Presentation

Veer (Lead)

- Write `run_all.py`: sequential execution of all module scripts
- Add `argparse`: `--mode [train|predict|dashboard]`
- Final GitHub push — tag release v1.0
- README: architecture diagram, setup steps, usage examples
- Record 3-min demo video of dashboard walkthrough

Deliverable: `run_all.py`

All Members

- Each member writes a 1-page module summary (limitations, assumptions, future scope)
- Assemble slide deck: Problem → Architecture → Results → Demo → Next Steps
- Rehearse live dashboard demo (5 minutes)
- Prepare Q&A;: expected questions on model accuracy, data privacy, scalability

Deliverable: `final_presentation.pptx`

4. Module Deep-Dives — Technical Specifications

Module 2: Underwriting Risk Engine

Owner: Member 2

Parameter	Value
Algorithm	XGBoostClassifier (multi-class, softmax)
Target variable	risk_class: 0=Low, 1=Medium, 2=High, 3=VeryHigh
Key features	age, BMI, smoker_flag, condition_severity_score, claim_frequency
Hyperparameters	n_estimators=300, max_depth=6, learning_rate=0.1, subsample=0.8
Explainability	SHAP TreeExplainer — global bar chart + per-applicant waterfall
Output schema	applicant_id, risk_score (0-100), risk_class, shap_top3_drivers
Evaluation	AUC-ROC (OvR), F1-macro, confusion matrix

Module 3: Claim Cost & Pricing Engine

Owner: Member 3

Parameter	Value
Algorithm	XGBoostRegressor
Target variable	claim_cost (INR or USD — continuous)
Key features	risk_score (from Module 2), age, BMI, smoker_flag, hospitalisation_flag
Loading factors	Low risk: 1.15 Medium: 1.25 High: 1.45 Very High: 1.65
Output schema	applicant_id, predicted_claim_cost, loading_factor, recommended_premium
Evaluation	RMSE, MAE, R-squared, % within 10% of actual
Note	Premium must always >= floor(predicted_cost * 1.10) — hard constraint

Module 4: Portfolio Analytics Engine

Owner: Member 4

Parameter	Value
Claim frequency	Count of claims per 1,000 policies — segmented by region, age, product
Claim severity	Average claim cost — Plotly box chart by risk segment
Loss ratio	Total claim costs / Total premiums earned * 100 — target: 75-90%

Trend analysis	Rolling 3/6/12 month averages if temporal data available
KPI thresholds	Loss ratio > 100%: alert Frequency spike > 2 SD: flag
Output schema	segment, policy_count, avg_frequency, avg_severity, loss_ratio, trend
Visualisation	Plotly: grouped bar, line trend, heatmap by region x risk band

Module 5: Lapse Prediction Engine

Owner: Member 5

Parameter	Value
Algorithm	XGBoostClassifier (binary) + scale_pos_weight for imbalance
Target variable	lapsed: 1=churned within policy year, 0=retained
Key features	policy_tenure, premium_amount, claim_count, payment_delay_days, agent_interaction_count
Segments	At-Risk: prob > 0.70 Watch: 0.40-0.70 Stable: < 0.40
Retention actions	At-Risk: agent_call + discount Watch: loyalty_email Stable: no action
Output schema	applicant_id, lapse_probability, segment, recommended_action, shap_top3_drivers
Evaluation	AUC-ROC, Precision-Recall AUC (imbalanced class), Lift curve

5. AI Copilot — Implementation Guide

Architecture

The AI Copilot uses the **Claude API** (claude-sonnet model) to generate natural language actuarial insights. Portfolio metrics are serialised to a structured string and injected into the system prompt. The Streamlit page streams the response token-by-token using `st.write_stream`.

System Prompt Template

You are a senior actuarial analyst at a health insurer.
Your task: analyse the provided portfolio metrics and generate a concise executive summary.
Focus on: loss ratio trends, high-risk segments, lapse patterns, and premium adequacy.
Format: 3-5 bullet points, then a 2-sentence recommendation.
Tone: professional, data-driven, actionable.
Never fabricate numbers not present in the data provided.

Sample Python Integration

```
import anthropic, os
from dotenv import load_dotenv
load_dotenv()
client = anthropic.Anthropic(api_key=os.getenv('ANTHROPIC_API_KEY'))
def generate_summary(portfolio_metrics_str: str) -> str:
    message = client.messages.create(
        model='claude-sonnet-4-6',
        max_tokens=800,
        system=SYSTEM_PROMPT,
        messages=[{'role': 'user', 'content': portfolio_metrics_str}]
    )
    return message.content[0].text
```

Dashboard Pages (Streamlit)

Page	Content
Page 1: Risk Assessment	Input form → live risk score gauge (plotly indicator) + SHAP waterfall
Page 2: Premium Quote	Sliders for loading factor → premium table + affordability check
Page 3: Portfolio KPIs	Frequency/severity bar charts + loss ratio trend line + alert badges
Page 4: Lapse Heatmap	Segment heatmap + retention action table + exportable CSV
Page 5: AI Copilot	Text input for custom queries + streaming AI response + summary export

6. Deliverables & Acceptance Criteria

Deliverable	Owner	Due	Acceptance Criteria
processed_data.csv	Veer	Day 2	No nulls; all features scaled; train/test split columns present
applicant_risk_scores.csv	Member 2	Day 3	risk_score in [0,100]; risk_class in {0,1,2,3}; AUC-ROC >= 0.80
predicted_claim_cost.csv	Member 3	Day 4	No negative values; RMSE < 20% of mean claim cost
premium_quotes.csv	Member 3	Day 4	premium >= predicted_cost * 1.10 for all rows
portfolio_metrics.csv	Member 4	Day 4	loss_ratio column present; all KPIs documented
lapse_risk_scores.csv	Member 5	Day 4	lapse_probability in [0,1]; segment in {At-Risk, Watch, Stable}
executive_summary.txt	Veer (AI)	Day 6	Generated by Claude API; >= 3 actionable insights
XGBoost .pkl models	M2, M3, M5	Day 5	All models loadable via joblib.load(); predict() runs without error
Streamlit dashboard (app.py)	Veer	Day 6	streamlit run app.py launches without error; all 5 pages render
GitHub repo + README	Veer	Day 7	README has setup steps; run_all.py completes end-to-end in < 5 min

7. Risk Register & Mitigations

Level	Risk	Mitigation
High	Data unavailable on Day 1	Veer pre-downloads Kaggle dataset on Day 0. All members receive the CSV before sprint start.
High	Dashboard (Module 6) blocked by upstream delays	Veer builds Streamlit skeleton with mock CSV data on Day 6. Real outputs swapped in when ready.
Med	XGBoost overfitting on small dataset	Use 5-fold cross-validation. Member 6 benchmarks vs logistic regression. Early stopping on val loss.
Med	API cost overrun (Claude / OpenAI)	Cache responses locally. Use claude-haiku for dev; switch to sonnet only for final demo.
Med	Schema mismatch between modules	feature_dict.md agreed Day 0. Member 6 runs automated schema validation script on Day 5.
Low	SHAP computation too slow	Use TreeExplainer (10-100x faster than KernelExplainer). Subsample 1,000 rows for SHAP plots.
Low	Class imbalance in lapse model	Set scale_pos_weight = count_negative / count_positive. Evaluate with PR-AUC, not accuracy.

8. Team Responsibilities & Definition of Done

Member	Role	Responsibilities
Veer	Project Lead	Data cleaning, feature engineering, AI copilot, dashboard, integration, docs
Member 2	Underwriting	Risk classification (XGBoost), SHAP analysis, risk segmentation
Member 3	Pricing	Claim cost prediction, premium recommendation, model evaluation
Member 4	Portfolio Analytics	Frequency, severity, loss ratio, trend analysis, KPI charts
Member 5	Lapse Analytics	Lapse prediction, retention segmentation, driver analysis
Member 6	Validation	Model testing, benchmarking, schema checks, business insight review

Definition of Done (DoD)

- All 10 deliverable files exist in /outputs/ or /models/ and are non-empty
- Every model loads and runs predict() without error on a new random sample
- Streamlit dashboard runs end-to-end with streamlit run app.py
- GitHub repo has a tagged release v1.0 with all files committed
- README contains setup, usage, and known limitations
- executive_summary.txt contains at least 3 data-backed insights from the AI copilot
- Presentation slide deck covers: problem, architecture, demo, results, limitations

Daily Sync Protocol

Each day at **6:00 PM IST**, all members post a brief update in the shared channel:

- What I completed today
- What I'm doing tomorrow
- Any blockers (tag @Veer if blocking integration)

Veer reviews and updates the shared GitHub project board after each sync.

9. Future Scope & Production Roadmap

Phase 2: Production Hardening (Month 2-3)

- Replace Streamlit with FastAPI backend + React frontend for production scale
- Add model retraining pipeline: automated weekly refit on new claims data
- Implement model drift detection: alert when feature distribution shifts > threshold
- Add audit trail: log every underwriting decision with timestamp and model version
- Connect to real claims management system via REST API or database connector

Phase 3: Advanced Capabilities (Month 4-6)

- Survival analysis for lapse modelling: time-to-lapse instead of binary flag
- GLM / credibility theory layer on top of ML pricing for regulatory compliance
- Multi-line portfolio: extend from health to motor and life
- Real-time scoring API: underwrite an applicant in < 200ms
- Regulatory reporting module: auto-generate IRDAI/Solvency II compliant reports